

Relatório 7 - Prática: Integração com Banco de Dados usando FastAPI (III)

José Carlos Seben de Souza Leite

Descrição da atividade

Iniciando este novo card trabalhando com a integração de banco de dados utilizando FastAPI fizemos a instalação de algumas ferramentas que vamos utilizar na elaboração deste card sendo elas o **sqlalchemy**, e o **alembic**.

Antes de explicar cada um, é interessante explicar um conceito que vamos utilizar que é o **ORM (Object Relational Mapper)** que basicamente serve para fazer os mapeamentos e registros de um banco de dados, assim nos permitindo trabalhar com um banco de dados em Python, por exemplo, sem a necessidade de estarmos fazendo códigos em SQL puro dentro de nosso projeto, o ORM atua como uma espécie de “tradutor”.

Ficando nossas ferramentas responsáveis por:

- **Sqlalchemy:** Vai ser a nossa ferramenta de ORM, ou seja, nosso tradutor para nos comunicarmos com o banco de dados em Python.
- **Alembic:** Uma biblioteca que serve como um tipo de versionamento para nosso banco de dados, ao fazermos modificações o alembic lê estas modificações e as adiciona ao banco de dados.

Após conhecermos um pouco sobre as ferramentas que utilizamos podemos ir para onde criamos nossa primeira tabela no nosso bando de dados a tabela user:



```
1 @table_registry.mapped_as_dataclass
2 class User:
3     __tablename__ = 'users'
4
5     id: Mapped[int] = mapped_column(init=False, primary_key=True)
6     username: Mapped[str] = mapped_column(unique=True)
7     password: Mapped[str]
8     email: Mapped[str] = mapped_column(unique=True)
9     created_at: Mapped[datetime] = mapped_column(
10         init=False, server_default=func.now()
11     )
12
```

Aqui onde basicamente criamos uma tabela onde teoricamente manteria o perfil de nossos usuários e usamos definições de SQL através da função `mapped_column` para, por exemplo, na primeira linha a iniciar falso assim atribuindo ao banco de dados a preencher e não ao usuário, e a adicionando como chave primaria da tabela. Outra parte interessante deste bloco de código é as duas ultimas linhas onde definimos um campo em nossa tabela para registrar quando foi criado o registro, ou seja, quando aquele perfil que estamos procurando foi criado, iniciando ela como false assim designando a responsabilidade para que o próprio banco de dados preencha a mesma, utilizando a função de `func.now()` pegando a hora exata de criação do usuário (Utilizando a hora do banco de dados).

Seguindo o padrão do curso sempre após fazermos uma nova função criamos um teste para a mesma e aqui não foi diferente, mas antes de criarmos o teste fizemos um passo adicionando uma configuração para testes no nosso [confest.py](#).

```
1 @pytest.fixture
2 def session():
3     engine = create_engine('sqlite:///memory:')
4     table_registry.metadata.create_all(engine)
5
6     with Session(engine) as session:
7         yield session
8
9     table_registry.metadata.drop_all(engine)
```

Essa configuração que impede que armazenemos lixo no nosso banco de dados com testes, com a linha **engine = create_engine('sqlite:///memory:')** criamos um banco de dados temporário na nossa memória ram não utilizando o banco de dados real, permitindo após terminar o teste limpar o mesmo com `drop_all(engine)` nos possibilitando rodar testes mais rápidos por rodar na memória ram e não armazenar lixo no nosso banco de dados real.

Com essa configuração de um banco de dados temporário para testes construímos nosso teste de criar um usuário.

```
1 def test_create_user(session):
2     new_user = User(
3         username='test_user', password='test_pass', email='test@example.com'
4     )
5
6     session.add(new_user)
7     session.commit()
8
9     user = session.scalar(select(User).where(User.username == 'test_user'))
10
11     assert user.username == 'test_user'
```

Onde basicamente ele cria um novo usuário no banco, passando os dados que necessitam ser preenchidos e salva esse novo usuário, após isso ele faz um select procurando por exatamente um usuário com o nome que passou no username testando se aquele usuário foi realmente criado e armazenado no banco de dados.

Concluindo esta parte do card finalizamos fazendo uma migração, que basicamente é quando fazemos alguma alteração no nosso banco de dados por código o alembic responsável por as migrações reconhece estas e as aplica no banco de dados com um ID de alteração, sendo um tipo de versionamento para nosso banco de dados.

Iniciando a parte no card sobre **Integração do banco de dados com FastAPI** é aqui que abandonaremos nossa simulação de um banco de dados e vamos migrar para um banco de dados real.

Um conceito que utilizamos para fazer essa migração é o **session** do **Sqlalchemy** que podemos ver ele como um responsável por cuidar das nossas ações em códigos para se comunicar com o banco de dados, como sua principal função é englobar as alterações que fazemos em “conjuntos” e as aplicar diretamente no banco funcionando como algo parecido a um carrinho onde podemos fazer diversas operações e as mandar todas de uma vez só para o banco de dados com o código **session.commit()** e para fazer a atualização dos objetos gerados no banco de dados **session.refresh()**.

Outro conceito que utilizamos para a nossa migração agora é algo do **FastAPI** chamado **depends** que é basicamente uma tecnologia que utilizamos, por exemplo, quando vamos declarar uma função é a mesma recebe como parâmetro a saída de outra função, utilizamos o **depends** para a função que depende da outra não executar e esperar a outra executar e mandar para ela seu parâmetro para daí, sim, ela ser executada. Esse conceito no **FastAPI** é chamado de injeção de dependência sendo um dos conceitos principais do **FastAPI**.

Com a migração da nossa simulação de um BD para um real tivemos que adaptar todas nossas funções e testes para agora operar com um BD real.

```
1 @app.put(
2     '/users/{user_id}', status_code=HTTPStatus.OK, response_model=UserPublic
3 )
4 def update_user(
5     user_id: int, user: UserSchema, session: Session = Depends(get_session)
6 ):
7
8     db_user = session.scalar(select(User).where(User.id == user_id))
9
10    if not db_user:
11        raise HTTPException(
12            status_code=HTTPStatus.NOT_FOUND, detail='User not found!'
13        )
14
15    db_user.email = user.email
16    db_user.username = user.username
17    db_user.password = user.password
18
19    session.add(db_user)
20    session.commit()
21    session.refresh(db_user)
22
23    return db_user
24
```

A função acima é um exemplo de utilização tanto do **depends** quanto do **session**, o **depends** sendo utilizado como parâmetro de session aguardando a resposta da função `get_session` para daí fazer a injeção de dependências na outra função assim permitindo que ela seja executada,

e session sendo utilizado para adicionar o usuário, mandar para o BD e receber a resposta no código.

Entrando no último conteúdo do nosso card sendo sobre autenticação e autorização, trabalhando deis de a liberação que um usuário pode ou não pode realizar até criptografia e armazenamento de senhas.

Para isso fizemos a instalação de algumas bibliotecas novas sendo elas:

- **pwdlib[argon2]:** utilizada para realizar o hashing de senhas.
- **pyjwt:** para geração de JWT.

Nos utilizamos o hashing nas senhas para com a senha inserida pelo usuário criar um hash aleatório e salvar o mesmo assim ninguém tendo acesso à senha do mesmo, e no caso de um vazamento de dados, por exemplo, a senha que vai vazar será esse hash, pois é o que armazenamos e nunca a senha real digitada, então quando o usuário for logar ele digitara sua senha e o sistema terá que a comparar com a “senha suja” que armazenamos.

Outro recurso utilizado nesse card foi o JWT (Json Web Token) gerado por nossa biblioteca **pyjwt** que são tokens temporários gerados para mantermos a segurança e controle de autenticações, esses tokens são gerados com tempos configuráveis no sistema para que uma vez que o usuário logar gerar esse token e pelo tempo determinado suas requisições vão ser autenticadas por esse mesmo, livrando o usuário de toda requisição que fazer precisar logar novamente no sistema.

Com esse sistema de login funcionando e os **JWTs** pudemos configurar as permissões e restrições que são o que impede de um usuário José deletar, atualizar ou qualquer outra coisa da conta de outra pessoa, essas permissões são o que definem as ações e permissões que cada usuário terá normalmente dividido por níveis de permissão dependendo do contexto do sistema.

Como o código pessoal foi uma junção de dois projetos, utilizando para o front end o projeto de dashboard realizado no card 5 como código pessoal, e o back end foi utilizando o código desenvolvido durante o card 7.

Utilizando o Docker para facilitar a rodar front/back simultaneamente, tivemos que fazer algumas adaptações no código para funcionar corretamente, como no front end estava funcionando independente com um sistema de login “hipotético”, e um “banco de dados” no localStorage, modificamos isso para ele se comunicar com o back e fazer requisições para o mesmo como na função a baixo.

```

1  async createSurvey(input: SurveyInput): Promise<SurveyResponse> {
2      const item = await apiClient<ApiSurveyDTO>("/surveys/", {
3          method: "POST",
4          headers: this.getHeaders(),
5          body: JSON.stringify({
6              readingDate: input.readingDate.toISOString(),
7              theme: input.theme,
8              frequency: input.frequency,
9          }),
10     })
11
12     return {
13         id: String(item.id),
14         readingDate: new Date(item.readingDate),
15         theme: item.theme,
16         frequency: item.frequency,
17     }
18 }

```

Função responsável por receber os dados de **SurveyInput** os formatar no esquema de JSON que utiliza para conexão com o back e fazer o envio HTTP com o método POST para criação de dados no banco, a autenticação é realizada por **this.getHeaders()** que anexa o token salvo no navegador ao envio.

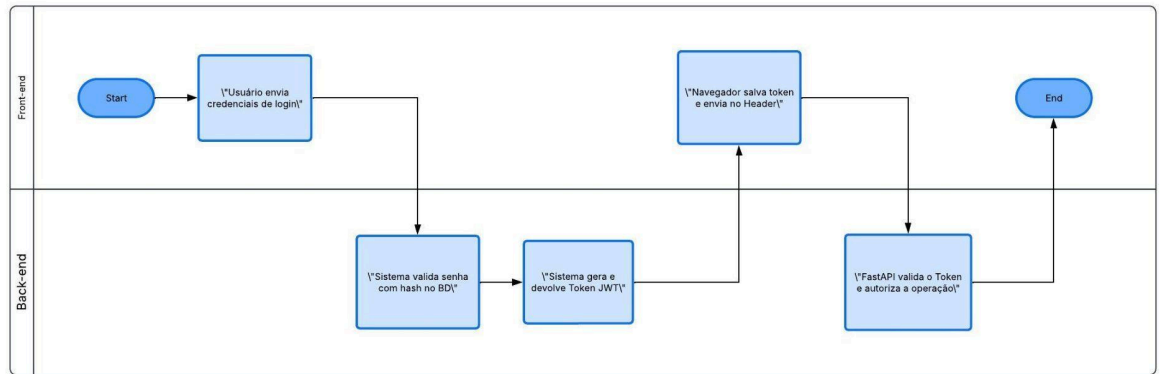
Essa requisição vai ser recebida pelo back end e processada pela seguinte função:

```

1  @app.post(
2      '/surveys/', status_code=HTTPStatus.CREATED, response_model=SurveyPublic
3  )
4  def create_survey(
5      survey: SurveySchema,
6      session: Session = Depends(get_session),
7      current_user=Depends(get_current_user),
8  ):
9      db_survey = Survey(
10         readingDate=survey.readingDate,
11         theme=survey.theme,
12         frequency=survey.frequency,
13     )
14     session.add(db_survey)
15     session.commit()
16     session.refresh(db_survey)
17     return db_survey

```

Que ao receber os dados enviados pelo front em JSON e faz a verificação da autenticação enviada pelo front se der certo converte os dados recebidos para o formato do banco de dados e faz a inserção no mesmo, após realizar esse processo caso tudo de certo retorna uma resposta para o front de sucesso.



Essas duas foram um exemplo de como agora está funcionando nosso sistema com as novas funções ou as antigas adaptadas a nova rotina.

Dificuldades

Na parte do desenvolvimento do curso não tive muita dificuldade, a maior dificuldade foi na hora da integração com o front end no projeto próprio.

Conclusões

Com esse card e o projeto desenvolvido consegue criar uma base tanto de uso de ORM quanto de conexão entre front e back.

Referencias

https://www.youtube.com/live/_87z5b4szW4?si=NMHnCLWJZHicMKBh
<https://www.youtube.com/live/6pXA6zg6hT0?si=4RRPBw1PK8mUATQ5>
<https://www.youtube.com/live/STt-IARdLSM?si=dpAbAiPTXlrWKIk6>